

DOSSIER S1 — CADRAGE & ARCHITECTURE

FinData Solutions — Pipeline Big Data Hadoop

Module DEPE855 | EPSI Mastère Expert en Ingénierie des données

1. MATRICE DES BESOINS

Flux	Source technique	Fréquence	Volume estimé	Équipe(s) destinataire(s)	Format de sortie attendu	Partitionnement HDFS	Contrainte de sécurité
Flux 1 – Transactions bancaires	PostgreSQL, table <code>transactions</code> (mise à jour quotidienne)	Quotidienne (extraction J, lancée à 06h30)	~500 000 lignes/jour (~2 Go brut CSV)	Data Science, B.I., Conformité & Audit	Parquet (compression Snappy) — agrégé par <code>code_banque</code> et jour	<code>/production/</code> <code>Pseudonymisation/annee=</code>	obligatoire : <code>iban</code> → SHA-256 + sel fixe ; <code>client_id</code> → identifiant séquentiel anonyme. Conformité RGPD + DSP2.
Flux 2 – Cours boursiers	Fichiers CSV déposés via SFTP (<code>/data/courses/avant-AAA-MM-JJ.csv</code>)	Quotidienne (dépôt avant 06h00, avant Flux 1)	~500 à 1 000 lignes/fichier (~50 Ko)	Référentiel interne utilisé par le Flux 1 uniquement	Parquet (compression Snappy)	<code>/ref/cours/</code> <code>Filter=AAAA-</code>	source : seuls ECB, Bloomberg, Reuters sont acceptés. Pas de DCP.
Flux 3 – Logs applicatifs	Flux JSON semi-structuré collecté par Apache NiFi vers zone de staging	Continue (collecte temps réel), traitement batch quotidien à 07h00	~200 000 événements/jour (~500 Mo JSON)	Équipe technique (exploitation), Data Science	Parquet (compression Snappy) — comptage erreurs par service et par heure	<code>/production/</code> <code>Aucun service=XXX/da</code>	donnée à caractère personnel directe ; filtrage strict niveau ERROR et CRITICAL uniquement.

Dépendance critique : Le Flux 2 doit être intégralement traité (zone `/ref/cours/`) **avant** le lancement du Flux 1, car la conversion de devises en EUR requiert les taux de clôture du jour.

2. SCHÉMA GLOBAL DU PIPELINE (DIAGRAMME DE FLUX)

[illegible]

Zones HDFS définies

Zone	Chemin	Rôle	Rétention
Raw	<code>/raw/transactions/</code> , <code>/raw/courses/</code>	Données brutes post-ingestion, non transformées	7 jours
Staging	<code>/staging/logs/</code>	Logs JSON en attente de traitement Spark	3 jours
Référentiel	<code>/ref/cours/</code>	Taux de change de clôture validés	5 ans
Production	<code>/production/transactions/</code> , <code>/production/logs/</code>	Données transformées, pseudonymisées, prêtes à la consommation	5 ans

Dépendances entre flux (séquencement Airflow)

```
graph TD
    ingest_flux2_sftp --> transform_flux2
    ingest_flux1_sqoop --> ingest_flux3_nifi_check
    ingest_flux3_nifi_check --> transform_flux1
    transform_flux1 --> transform_flux3
    transform_flux3 --> notify_success
```

3. JUSTIFICATION DES CHOIX TECHNIQUES

Ingestion — Flux 1 : Apache Sqoop

Choix retenu : Apache Sqoop avec lecture JDBC depuis PostgreSQL.

Justification : Sqoop est l'outil de référence pour le transfert de données entre bases relationnelles et HDFS dans l'écosystème Hadoop. Il prend nativement en charge JDBC/PostgreSQL, gère le parallélisme des mappers (`-m 4`) pour accélérer l'extraction des 500 000 lignes quotidiennes, supporte l'export incrémental (`--incremental append`), et produit directement du Parquet via `--as-parquetfile`. Alternative NiFi non retenue pour ce flux car NiFi ajouterait une couche de complexité inutile sur une extraction batch simple et planifiable.

Ingestion — Flux 2 : Script Shell + `hdfs dfs -put`

Choix retenu : Script Shell (bash) combinant SFTP (`sftp -i key`) et `hdfs dfs -put`.

Justification : Le volume est minimal (~50 Ko/fichier) et le cas d'usage est trivial (1 fichier par jour). Un script shell de 15 lignes est suffisant, maintenable, et ne nécessite pas de licence ou de démarrage d'un service supplémentaire. NiFi ou Kafka seraient surdimensionnés pour ce flux.

Ingestion — Flux 3 : Apache NiFi

Choix retenu : Apache NiFi avec processors `GetFile` (depuis répertoire de staging) ou `ListenHTTP`.

Justification : Le flux JSON semi-structuré requiert un outil capable de gérer la variabilité du schéma, le routage conditionnel et l'accumulation en zone de staging. NiFi offre une interface graphique de monitoring du flux, des processors natifs JSON, et une intégration HDFS via `PutHDFS`. C'est l'outil recommandé par le sujet pour ce type de source.

Transformation : PySpark (Python)

Choix retenu : PySpark 3.x (Python).

Justification : L'équipe Data Science travaille déjà en Python avec PySpark — utiliser le même langage réduit la friction, facilite les revues de code et le transfert de compétences. PySpark permet des UDFs Python pour des transformations métier complexes. Spark Scala aurait de meilleures performances JVM natives, mais l'équipe n'a pas de compétences Scala déclarées. PySpark avec compilation Catalyst est suffisant pour 500K lignes/jour.

Orchestration : Apache Airflow

Choix retenu : Apache Airflow avec `SparkSubmitOperator` et `BashOperator`.

Justification : Airflow est le standard industrie pour l'orchestration de pipelines de données. Les DAGs sont définis en Python, le suivi des dépendances entre tâches (Flux 2 avant Flux 1) est natif, le retry automatique est configurable, et l'interface web permet de visualiser l'historique d'exécution. Apache Oozie est lié à l'écosystème CDH (Cloudera) et plus verbeux (XML). Cron est trop limité (pas de gestion des dépendances, pas de retry).

Format de fichier : Parquet avec compression Snappy

Choix retenu : Apache Parquet avec codec Snappy.

Justification : Parquet est un format **columnar** — seules les colonnes nécessaires à la requête sont lues (projection pushdown), ce qui réduit les I/O de 60 à 90% par rapport à CSV. Il supporte les statistiques de colonnes (min/max) permettant le **predicate pushdown** (saut de fichiers entiers non pertinents). Snappy offre un bon équilibre compression/vitesse de décompression (pas de CPU bloquant). ORC est légèrement supérieur pour les requêtes Hive/Tez mais moins bien supporté par les outils BI (PowerBI) et par les bibliothèques Python des data scientists.

4. DATASETS SÉLECTIONNÉS

Les datasets sont **générés synthétiquement** via le script `generate_datasets.py` (livré en S2) car l'environnement de test ne dispose pas d'un PostgreSQL de production ni d'un SFTP réel.

Flux	Dataset	Générateur	Volume	Fichier produit
Flux 1	Transactions bancaires synthétiques (Python Faker)	<code>generate_datasets.py</code>	550 000 lignes (dont ~2% FRAUD, ~1% à rejeter)	<code>test_data/transactions_202</code>
Flux 2	Taux de change EUR (sources mixtes simulées)	<code>generate_datasets.py</code>	800 lignes (30 devises × horodatages + sources invalides)	<code>test_data/courses_2024-01-</code>
Flux 3	Logs applicatifs JSON semi-structurés	<code>generate_datasets.py</code>	220 000 lignes JSON (tous niveaux, ~15% ERROR/CRITICAL)	<code>test_data/logs_2024-01-15.</code>

Simulation de montée en charge (S3) : le script dispose d'un paramètre `--double` qui génère le double du volume (1,1 million de transactions, ~440 000 logs ERROR/CRITICAL).



5. RÉPONSES AUX QUESTIONS DE RÉFLEXION (S1)

Question 1 — Pourquoi le Flux 2 doit-il être exécuté avant le Flux 1 ? Gestion de l'absence du fichier CSV

Réponse :

Le Flux 1 applique une règle métier de **conversion de tous les montants en EUR** en utilisant les taux de change du jour. Ces taux proviennent exclusivement du référentiel `/ref/cours/` alimenté par le Flux 2. Si le Flux 2 n'a pas encore été exécuté au moment où le Flux 1 démarre, la jointure `df_transactions.join(df_cours, on="devise")` produira des valeurs `NULL` pour `montant_eur` sur l'ensemble des transactions en devises étrangères — rendant les agrégats inexploitable par la cellule B.I. et le Conformité & Audit.

Gestion de l'absence du fichier CSV :

Dans le DAG Apache Airflow, on intercale un **sensor** avant le job Spark du Flux 2 :

```
from airflow.sensors.hdfs_sensor import HdfsSensor from airflow.operators.python import
PythonOperator check_sftp_file = HdfsSensor( task_id='check_cours_sftp_disponible',
filepath='/raw/courses/{{ ds }}.csv', hdfs_conn_id='hdfs_default', timeout=7200, # Attente
max 2 heures poke_interval=300, # Vérification toutes les 5 minutes mode='reschedule', #
Libère le worker pendant l'attente )
```

Stratégie de fallback si dépassement du timeout :

1. L'alerte Airflow notifie l'équipe par email/Slack.
2. Une tâche de fallback (`use_previous_day_cours`) copie le référentiel de la veille (`/ref/cours/date_partition={{ yesterday_ds }}`) vers la date du jour en le taggant `source=FALLBACK`.
3. Le Flux 1 démarre avec les taux de la veille (acceptable réglementairement pour les conversions intra-journalières selon DSP2).
4. Le run sera corrigé manuellement via un **backfill Airflow** (`airflow dags backfill`) dès que le fichier est disponible.

Question 2 — La pseudonymisation SHA-256 avec sel fixe est-elle réversible ? Risques et alternatives

Réponse :

SHA-256 est une fonction de hachage à sens unique (one-way function) — elle n'est pas réversible mathématiquement. Il est impossible de retrouver l'IBAN original à partir du seul hash. Cependant, cette protection présente une **vulnérabilité majeure** :

L'espace des IBANs français est fini et structuré (**FR76** + 23 chiffres). Un attaquant disposant du sel peut construire une **table arc-en-ciel (rainbow table)** ou réaliser une **attaque par dictionnaire** en calculant le hash de tous les IBANs connus. Avec un sel fixe identique pour tous les enregistrements, si **un seul IBAN est compromis** (ex: attaque corrélation sur données bancaires), le sel peut être inféré, exposant potentiellement l'intégralité des 8 millions de transactions quotidiennes.

Risques concrets si le sel est compromis :

- Réidentification de tous les clients sur toute l'historique (5 ans selon la contrainte Conformité).
- Violation grave du RGPD (article 25 — Privacy by Design) pouvant entraîner une amende jusqu'à 4% du CA annuel mondial.
- Non-conformité DSP2 sur la protection des données de paiement.

Alternative recommandée — Tokenisation via vault sécurisé :

- Au lieu du SHA-256 sel fixe, utiliser **HashiCorp Vault (Transit Secrets Engine)** ou **AWS KMS** :
- L'IBAN est envoyé au vault qui retourne un token opaque aléatoire.
 - La correspondance token ↔ IBAN est stockée dans le vault, accessible **uniquement** par le service Conformité & Audit avec authentification forte.
 - Le sel/clé est rotaté régulièrement sans impacter les données déjà pseudonymisées.
 - Alternative plus légère : **HMAC-SHA-256 avec sel aléatoire par enregistrement** (stocké en colonne `sel_hash` dans une table séparée chiffrée).

Question 3 — Avantage du format Parquet par rapport à CSV pour les requêtes ad hoc de la cellule B.I.

Réponse :

Le format CSV est **orienté ligne** : pour lire une seule colonne, il faut parcourir toutes les colonnes de chaque ligne. Le format Parquet est **orienté colonne** (columnar) : les valeurs d'une même colonne sont stockées contiguës sur le disque.

Avantages concrets pour la cellule B.I. (requêtes PowerBI/Spark SQL) :

Critère	CSV	Parquet
Requête <code>SELECT code_banque, SUM(montant_eur)</code> sur 500K lignes	Lit 100% des données (~2 Go)	Lit seulement 2 colonnes (~120 Mo) → 94% d'I/O économisés
Filtrage <code>WHERE annee=2024 AND mois=1</code>	Scan complet du fichier	Partition pruning : seuls les répertoires <code>annee=2024/mois=01/</code> sont lus
Compression	Aucune native, taille ~2 Go	Snappy, taille ~400 Mo (-80%)
Typage des colonnes	Tout est string, inférence coûteuse	Schéma embarqué, types natifs (LongType, DoubleType, TimestampType)
Predicate pushdown	Impossible	Statistiques min/max par colonne par bloc : PowerBI peut sauter des blocs entiers

Compatibilité PowerBI	Via connecteur CSV (lent)	Connecteur Parquet natif, lecture optimisée
-----------------------	---------------------------	---

Exemple chiffré : La cellule B.I. veut le taux de fraude par banque sur le mois de janvier 2024.

- **Avec CSV** : Spark lit ~60 Go de données (31 jours × ~2 Go), durée ~8 minutes.

- **Avec Parquet partitionné** : Spark lit uniquement les fichiers du répertoire `annee=2024/mois=01/`, soit ~3 colonnes × ~15 millions de lignes ≈ **600 Mo**, durée **~25 secondes**.

Le format Parquet réduit ainsi les coûts de calcul YARN, accélère les dashboards PowerBI, et améliore l'expérience des analystes B.I.